

NestJS

Résumé complet et pédagogique

Un guide clair et structuré pour comprendre le framework NestJS : son architecture, ses concepts fondamentaux, ses avantages, et des exemples de code commentés pour bien démarrer.

Sommaire

1. Qu'est-ce que NestJS ? 3
2. Pourquoi utiliser NestJS ? 4
3. Architecture générale 5
4. Les concepts fondamentaux 6
5. Exemple de code complet 9
6. Fonctionnalités avancées 11
7. NestJS vs Express / autres frameworks 12
8. Écosystème et outils 13
9. Conclusion 14

1 Qu'est-ce que NestJS ?

NestJS est un framework **Node.js** progressif, écrit en **TypeScript**, utilisé pour construire des applications **backend** (côté serveur) efficaces, fiables et facilement évolutives. Il a été créé en 2017 par **Kamil Myśliwiec**.

NestJS ne réinvente pas la roue : il s'appuie par défaut sur **Express.js** (et peut aussi utiliser **Fastify**), mais il ajoute par-dessus une architecture complète, inspirée d'**Angular**, avec des concepts comme les modules, les injections de dépendances, les décorateurs, etc.

> **En une phrase**
>
> NestJS, c'est un cadre de travail (framework) qui apporte de l'ordre, de la structure et des bonnes pratiques à vos applications Node.js, un peu comme Angular le fait côté frontend.

Les piliers de NestJS

Pilier	Description
TypeScript	Langage principal, apporte le typage statique et réduit les erreurs
Programmation orientée objet	Classes, décorateurs, héritage
Programmation fonctionnelle	Compatible avec RxJS et les flux réactifs
Injection de dépendances	Gestion automatique des dépendances entre composants
Modularité	Organisation du code en modules réutilisables

2 Pourquoi utiliser NestJS ?

Quand on démarre avec Node.js et Express, on est souvent livré à soi-même pour organiser son code. NestJS répond à ce problème en imposant une **architecture claire** et en facilitant la maintenance sur le long terme.

Avantages

- Architecture claire et standardisée
- Excellent support de TypeScript
- Idéal pour les gros projets d'entreprise
- Facile à tester (DI intégrée)
- Écosystème riche (auth, DB, GraphQL, microservices...)
- Documentation officielle très complète

Points d'attention

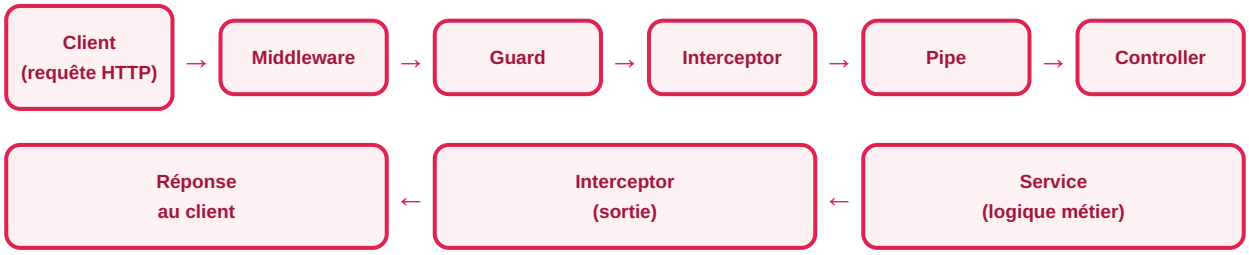
- Courbe d'apprentissage plus élevée qu'Express seul
- Peut sembler "lourd" pour de très petits projets
- Nécessite de bien comprendre la DI et les décorateurs

Cas d'usage typiques

- **API REST** robustes pour applications web/mobiles
- **API GraphQL**
- **Microservices** (avec TCP, Redis, RabbitMQ, Kafka...)
- Applications **temps réel** avec WebSockets
- Applications **CLI** (en ligne de commande)

3 Architecture générale

NestJS repose sur une architecture en couches, où chaque brique a un rôle précis. Voici le flux typique d'une requête HTTP :



Structure d'un projet NestJS

```
src/
├── main.ts                 # Point d'entrée de l'application
├── app.module.ts            # Module racine
├── app.controller.ts        # Contrôleur racine
├── app.service.ts           # Service racine
└── users/
    ├── users.module.ts       # Module "users"
    ├── users.controller.ts   # Routes liées aux utilisateurs
    ├── users.service.ts      # Logique métier
    ├── dto/
    │   └── create-user.dto.ts
    └── entities/
        └── user.entity.ts
```

Chaque **module** encapsule un ensemble cohérent de fonctionnalités (contrôleurs, services, providers), ce qui rend l'application modulaire et facile à faire évoluer.

4 Les concepts fondamentaux

4.1 — Les Modules @Module

Un module est une classe annotée avec le décorateur `@Module()`. Il regroupe des contrôleurs et des services liés entre eux. Chaque application a au moins un module racine (`AppModule`).

```
import { Module } from '@nestjs/common';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';

@Module({
  controllers: [UsersController],
  providers: [UsersService],
  exports: [UsersService], // rend le service disponible ailleurs
})
export class UsersModule {}
```

4.2 — Les Contrôleurs @Controller

Les contrôleurs gèrent les **requêtes entrantes** et renvoient des **réponses** au client. Ils définissent les routes de l'application.

```
import { Controller, Get, Post, Body, Param } from '@nestjs/common';
import { UsersService } from './users.service';

@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Get()
  findAll() {
    return this.usersService.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.usersService.findOne(id);
  }

  @Post()
  create(@Body() userData: any) {
    return this.usersService.create(userData);
  }
}
```

4.3 — Les Services (Providers) @Injectable

Un service contient la **logique métier**. Il est marqué avec `@Injectable()` pour pouvoir être injecté dans d'autres classes (contrôleurs, autres services).

```
import { Injectable } from '@nestjs/common';
```

```
@Injectable()
export class UsersService {
  private users = [];

  findAll() {
    return this.users;
  }

  findOne(id: string) {
    return this.users.find(u => u.id === id);
  }

  create(data: any) {
    const newUser = { id: Date.now().toString(), ...data };
    this.users.push(newUser);
    return newUser;
  }
}
```

4.4 — L'injection de dépendances (DI)

Plutôt que de créer manuellement une instance d'un service, NestJS s'en charge automatiquement grâce à son **conteneur d'injection de dépendances**. Il suffit de déclarer le type dans le constructeur.

> **À retenir**
>
> Vous **déclarez** ce dont vous avez besoin dans le constructeur, et NestJS se charge de **fournir** l'instance correspondante automatiquement. Cela rend le code plus testable et plus découplé.

4.5 — Les DTOs (Data Transfer Objects)

Un DTO définit la forme des données envoyées dans une requête, souvent combiné avec la librairie `class-validator` pour valider automatiquement les entrées.

```
import { IsString, IsEmail, IsInt, Min } from 'class-validator';

export class CreateUserDto {
  @IsString()
  name: string;

  @IsEmail()
  email: string;

  @IsInt()
  @Min(0)
  age: number;
}
```

4.6 — Middlewares, Guards, Pipes, Interceptors

Composant	Rôle	Exemple d'usage
Middleware	Exécuté avant que la requête n'atteigne le routeur	Logger, CORS

Guard	Détermine si une requête peut être traitée	Authentification, rôles
Pipe	Transforme ou valide les données entrantes	ValidationPipe, ParseIntPipe
Interceptor	Ajoute une logique avant/après l'exécution d'une méthode	Logging, mise en cache, transformation de la réponse
Exception Filter	Capture et formate les erreurs	Gestion centralisée des erreurs HTTP

```
// Exemple de Guard simple
@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    return !!request.headers.authorization;
  }
}
```

5 Exemple de code complet

Voici un mini projet illustrant l'assemblage de tous les concepts vus précédemment.

main.ts — Point d'entrée

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(3000);
}
bootstrap();
```

app.module.ts — Module racine

```
import { Module } from '@nestjs/common';
import { UsersModule } from './users/users.module';

@Module({
  imports: [UsersModule],
})
export class AppModule {}
```

users.controller.ts — Avec validation et DTO

```
@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Post()
  create(@Body() dto: CreateUserDto) {
    return this.usersService.create(dto);
  }

  @Get()
  findAll() {
    return this.usersService.findAll();
  }
}
```

> **▶ Résultat**
>
> Lancer `npm run start:dev` démarre l'application sur `http://localhost:3000`. Un `POST /users` avec un JSON valide crée un utilisateur ; un JSON invalide déclenche automatiquement une erreur **400 Bad Request** grâce au ValidationPipe.

6 Fonctionnalités avancées

6.1 — Bases de données (TypeORM / Prisma / Mongoose)

NestJS s'intègre facilement avec la plupart des ORM populaires via des modules dédiés comme `@nestjs/typeorm`, `@nestjs/mongoose` ou Prisma.

6.2 — Authentification avec Passport & JWT

Le module `@nestjs/passport` combiné à `@nestjs/jwt` permet de mettre en place facilement une authentification sécurisée basée sur des tokens.

6.3 — GraphQL

Avec `@nestjs/graphql`, on peut créer une API GraphQL en utilisant soit une approche *code-first* (décorateurs), soit *schema-first* (fichiers .graphql).

6.4 — Microservices

NestJS permet de transformer une application en microservice communiquant via TCP, Redis, NATS, RabbitMQ, Kafka ou gRPC, tout en gardant la même syntaxe de contrôleurs.

6.5 — WebSockets & temps réel

Grâce à `@nestjs/websockets`, il est facile de créer des applications en temps réel (chat, notifications) avec Socket.io ou WS.

6.6 — Tests

NestJS fournit un module de test intégré (`@nestjs/testing`) qui facilite l'écriture de tests unitaires et d'intégration grâce à l'injection de dépendances.

7 NestJS vs Express / autres frameworks

Critère	Express.js	NestJS	Fastify
Structure imposée	Non	Oui (modules, DI...)	Non
TypeScript natif	Non (ajouté manuellement)	Oui	Non (ajouté manuellement)
Courbe d'apprentissage	Faible	Moyenne à élevée	Faible
Performance brute	Bonne	Bonne (dépend du driver)	Excellente
Idéal pour	Petits projets, prototypes	Applications d'entreprise, gros projets	APIs très performantes
Écosystème intégré	Minimal	Très riche	Minimal

À noter

NestJS peut utiliser Express **ou** Fastify comme moteur HTTP sous-jacent. On profite ainsi de la performance de Fastify tout en gardant l'architecture de NestJS.

8 Écosystème et outils

NestJS CLI

L'outil en ligne de commande officiel permet de générer rapidement du code standardisé :

```
npm i -g @nestjs/cli
nest new mon-projet
nest generate module users
nest generate controller users
nest generate service users
```

Principaux packages officiels

Package	Utilité
`@nestjs/common`	Décorateurs et outils de base
`@nestjs/core`	Cœur du framework
`@nestjs/platform-express`	Adaptateur Express
`@nestjs/config`	Gestion des variables d'environnement
`@nestjs/swagger`	Génération automatique de documentation API
`@nestjs/typeorm`	Intégration TypeORM
`@nestjs/jwt`	Gestion des tokens JWT

9 Conclusion

NestJS est aujourd'hui l'un des frameworks backend Node.js les plus utilisés pour construire des applications **professionnelles, structurées et maintenables**. Sa proximité avec Angular en fait un excellent choix pour les équipes qui utilisent déjà TypeScript côté frontend.

Pour bien démarrer

1. Installer le CLI : `npm i -g @nestjs/cli`
2. Créer un projet : `nest new mon-app`
3. Explorer la structure générée
4. Créer votre premier module avec `nest generate resource`
5. Consulter la documentation officielle : **docs.nestjs.com**

Document généré comme résumé pédagogique sur NestJS.